Deadlock Avoidance and Detection Simulator using Banker's Algorithm

A PROJECT REPORT

Submitted by

Shivam Yadav
Chandesh Patwari
Rupesh Singh
Vineet Shukla

in partial fulfillment for the award of the degree of

Bachelor of Technology

IN

Computer Science and Engineering

SET, MRIIRS

MANAV RACHNA
vidyanataraksha

NAAC A++
ACCREDITED

ABSTRACT

This project presents the design and implementation of a simulator for deadlock avoidance and detection using Banker's Algorithm. Deadlock is a critical issue in operating systems where processes are unable to proceed due to resource contention, leading to system inefficiency.

The developed system evaluates whether the system is in a safe or unsafe state and determines a safe sequence for process execution. It also handles dynamic resource requests and detects deadlock conditions. The simulator is implemented using the C programming language and provides a detailed step-by-step execution of the algorithm.

The results demonstrate that Banker's Algorithm effectively prevents unsafe resource allocation and ensures smooth execution of processes without deadlock.

TABLE OF CONTENTS

1. Introduction
2. Results and Discussion
3. Conclusion
4. References

LIST OF TABLES

Table 1: Allocation Matrix

Table 2: Maximum Matrix

Table 3: Need Matrix

LIST OF FIGURES

Figure 1: Deadlock Condition

Figure 2: Flow of Banker's Algorithm

Figure 3: Safe Sequence Execution

Figure 4: Deadlock Detection Flow

Figure 5: Safe System

Figure 6: Unsafe System (Deadlock Possible)

Figure 7: Resource Granted

Figure 8: Resource Denied

Figure 9: Deadlock Detected

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

1. n: Number of processes
2. m: Number of resource types
3. Allocation: Resources currently allocated
4. Max: Maximum resource requirement
5. Need: Remaining resource requirement
6. Available: Available resources

INTRODUCTION

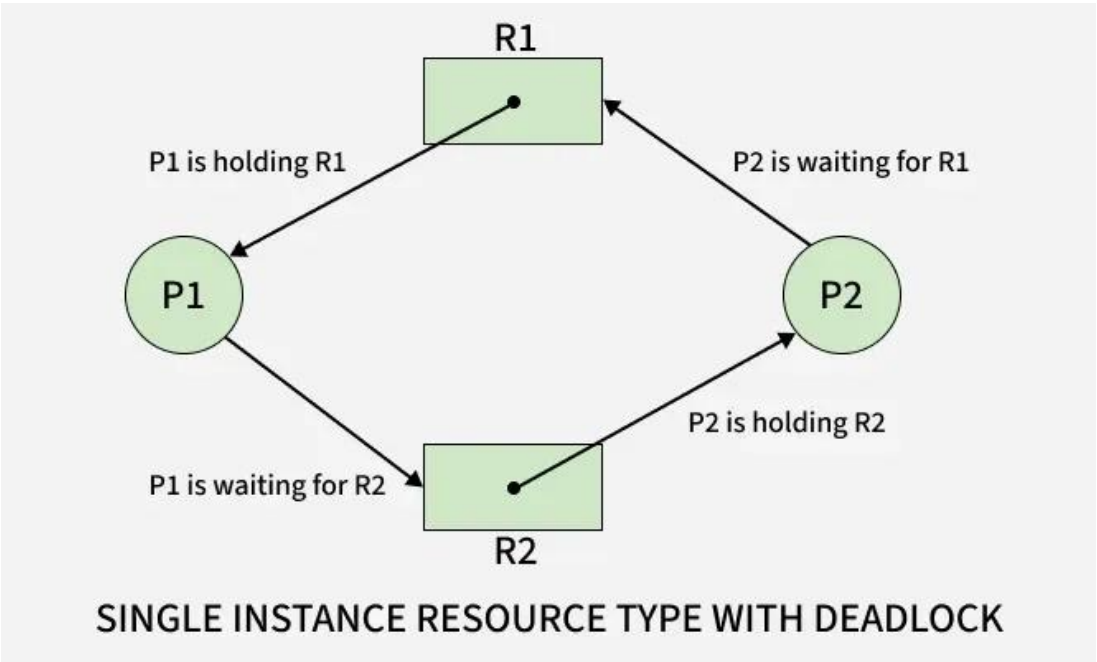

Figure 1: Deadlock Condition

Deadlock is a condition in which a set of processes are blocked because each process is holding at least one resource and waiting for another resource held by another process. This creates a circular wait condition, causing the system to halt indefinitely.

Operating systems use several techniques to deal with deadlocks, including prevention, avoidance, detection, and recovery. Among these, Banker's Algorithm is a widely used deadlock avoidance technique.

Banker's Algorithm ensures that the system always remains in a **safe state**. A safe state is defined as a condition in which there exists at least one sequence of process execution such that all processes can complete without leading to a deadlock.

This project implements a simulator that:

- Calculates the Need matrix
- Determines safe and unsafe states
- Handles dynamic resource requests
- Detects deadlock conditions

The implementation is carried out using the C programming language with a menu-driven interface, providing a clear and interactive way to understand resource allocation

RESULTS AND DISCUSSION

Input Data

Number of Processes: 5

Number of Resources: 3

Table 1: Allocation Matrix

0	**1**	**0**
2	**0**	**0**
3	**0**	**2**
2	**1**	**1**
0	**0**	**2**

Table 2: Maximum Matrix

7	**5**	**3**
3	**2**	**2**
9	**0**	**2**
2	**2**	**2**
4	**3**	**3**

Table 3: Available Resources

3	**3**	**2**

Need Calculation

Need = Max − Allocation

Example:

P1 Need = (3 2 2) − (2 0 0) = (1 2 2)

Safe State Analysis

The system evaluates processes step by step:

- Process P1 executes and releases resources
- Process P3 executes
- Process P4 executes
- Process P0 executes
- Process P2 executes

Safe Sequence:

```
Available = (3, 3, 2)

        ↓

[Check P1]
Need ≤ Available → YES
Execute P1
Release (2, 0, 0)
New Available = (5, 3, 2)

        ↓

[Check P3]
Need ≤ Available → YES
Execute P3
Release (3, 0, 2)
New Available = (8, 3, 4)

        ↓

[Check P4]
Need ≤ Available → YES
Execute P4
Release (2, 1, 1)
New Available = (10, 4, 5)

        ↓

[Check P0]
Need ≤ Available → YES
Execute P0
Release (0, 1, 0)
New Available = (10, 5, 5)

        ↓

[Check P2]
Need ≤ Available → YES
Execute P2
Release (3, 0, 2)
New Available = (13, 5, 7)

        ↓

SAFE SEQUENCE: P1 → P3 → P4 → P0 → P2
```

Figure 3: Safe Sequence Execution

Resource Request Handling

When a process requests resources:

1. The system checks if request ≤ need
2. Checks if request ≤ available
3. Verifies if system remains in safe state

If all conditions are satisfied, the request is granted. Otherwise, it is denied.

Deadlock Detection

```
                    START
                      ↓
       Input Allocation Matrix & Available Vector
                      ↓
          Initialize Work = Available
          Initialize Finish[i] = false
                      ↓

       ┌──────────────────────────────┐
       │ Find a process Pi such that: │
       │ Finish[i] = false AND        │
       │ Allocation[i] ≤ Work         │
       └──────────────────────────────┘

                      ↓
                ┌──────────────┐
                │ Process Found? │
                └──────────────┘

                  ↓        ↓
                YES       NO
                  ↓        ↓
  Work = Work + Allocation[i]   Exit Loop
  Finish[i] = true
                  ↓
            Repeat Step
                  ↓

      Check all Finish[i] values
                  ↓
       ┌──────────────────────────┐
       │ Any Finish[i] == false ? │
       └──────────────────────────┘

            ↓              ↓
           YES            NO
            ↓              ↓
  Deadlock Detected     No Deadlock
            ↓              ↓
  Display Processes       System Safe
  in Deadlock
```

Figure 4: Deadlock Detection Flow

If no process can proceed:

- The system identifies blocked processes
- Lists deadlocked processes

Discussion

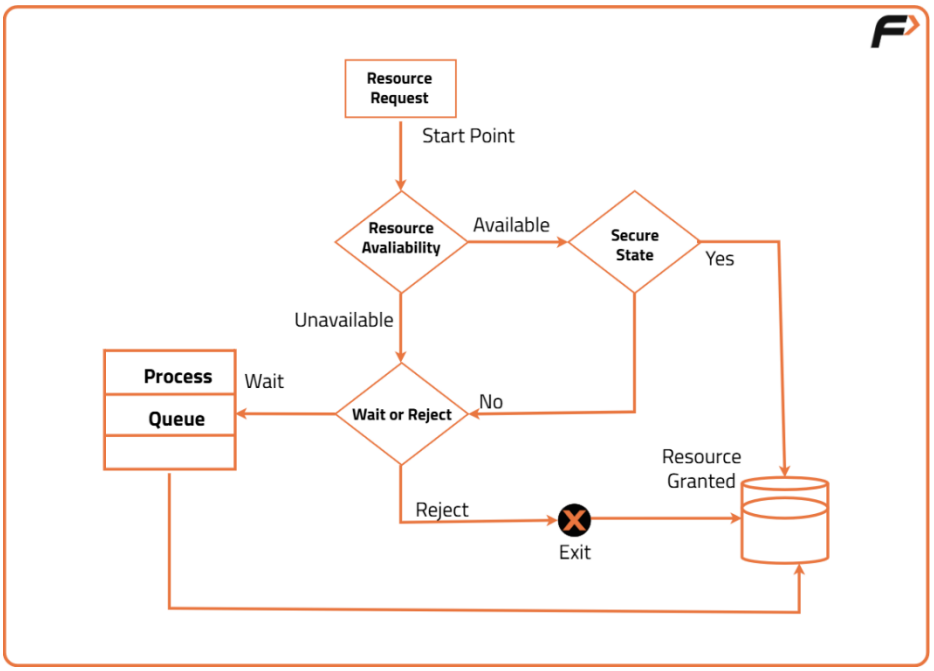

Figure 2: Flow of Banker's Algorithm

The implementation demonstrates Banker's Algorithm:

- Prevents unsafe allocations
- Maintains system stability
- Avoids deadlock situations

The step-by-step output improves understanding of system behavior.

DEMONSTRATION

```
PS D:\os> .\banker.exe
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

--- MENU ---
1. Show Matrices
2. Check Safe State
3. Request Resources
4. Detect Deadlock
5. Exit
Enter choice: 2

[RESULT] System is SAFE
Safe Sequence: P1 P3 P4 P0 P2

--- MENU ---
1. Show Matrices
2. Check Safe State
3. Request Resources
4. Detect Deadlock
5. Exit
Enter choice: |
```

Figure 5: Safe System

```
PS D:\os> .\banker.exe
Enter number of processes: 3
Enter number of resources: 2

Enter Allocation Matrix:
1 0
0 1
1 1

Enter Maximum Matrix:
2 1
1 2
2 2

Enter Available Resources:
0 0

--- MENU ---
1. Show Matrices
2. Check Safe State
3. Request Resources
4. Detect Deadlock
5. Exit
Enter choice: 2

[RESULT] System is UNSAFE (Deadlock possible)

--- MENU ---
1. Show Matrices
2. Check Safe State
3. Request Resources
4. Detect Deadlock
5. Exit
Enter choice: |
```

Figure 6: Unsafe System (Deadlock Possible)

```
Allocation:
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2

Maximum:
P0: 7 5 3
P1: 3 2 2
P2: 9 0 2
P3: 2 2 2
P4: 4 3 3

Need:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1
Available: 3 3 2

--- MENU ---
1. Show Matrices
2. Check Safe State
3. Request Resources
4. Detect Deadlock
5. Exit
Enter choice: 3

Enter process number: 1
Enter request vector: 1 0 2

[RESULT] System is SAFE
Safe Sequence: P1 P3 P4 P0 P2
[RESULT] Request GRANTED
```

Figure 7: Resource Granted

```
Allocation:
P0: 0 1 0
P1: 3 0 2
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2

Maximum:
P0: 7 5 3
P1: 3 2 2
P2: 9 0 2
P3: 2 2 2
P4: 4 3 3

Need:
P0: 7 4 3
P1: 0 2 0
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1
Available: 2 3 0

--- MENU ---
1. Show Matrices
2. Check Safe State
3. Request Resources
4. Detect Deadlock
5. Exit
Enter choice: 3

Enter process number: 0
Enter request vector: 3 3 0
[WAIT] Resources not available
```

Figure 8: Resource Denied

```
PS D:\os> .\banker.exe
Enter number of processes: 4
Enter number of resources: 2

Enter Allocation Matrix:
1 0
0 1
1 1
0 0

Enter Maximum Matrix:
2 1
1 2
2 2
1 1

Enter Available Resources:
0 0

--- MENU ---
1. Show Matrices
2. Check Safe State
3. Request Resources
4. Detect Deadlock
5. Exit
Enter choice: 4

Deadlocked Processes: P0 P1 P2 P3

--- MENU ---
1. Show Matrices
2. Check Safe State
3. Request Resources
4. Detect Deadlock
5. Exit
Enter choice: |
```

Figure 9: Deadlock Detected

CONCLUSION

This project successfully implements a deadlock avoidance and detection system using Banker's Algorithm. The simulator ensures that the system remains in a safe state by carefully managing resource allocation.

The results confirm that Banker's Algorithm is an effective method for preventing deadlocks in operating systems. The project also enhances understanding of process scheduling and resource management concepts.

Future improvements may include graphical interfaces and real-time simulations.

REFERENCES

1. Abraham Silberschatz, Peter B. Galvin – *Operating System Concepts*
2. Andrew S. Tanenbaum – *Modern Operating Systems*
3. GeeksforGeeks – Banker's Algorithm
4. Lecture Notes – Operating System